OS ORANGE 4

PES-VCS Lab Report

Operating Systems Lab

Name: Nirav Choudhary

SRN: PES2UG24CS323

GitHub Repository:

https://github.com/Nirav6769/PES2UG24CS323-pes-vcs

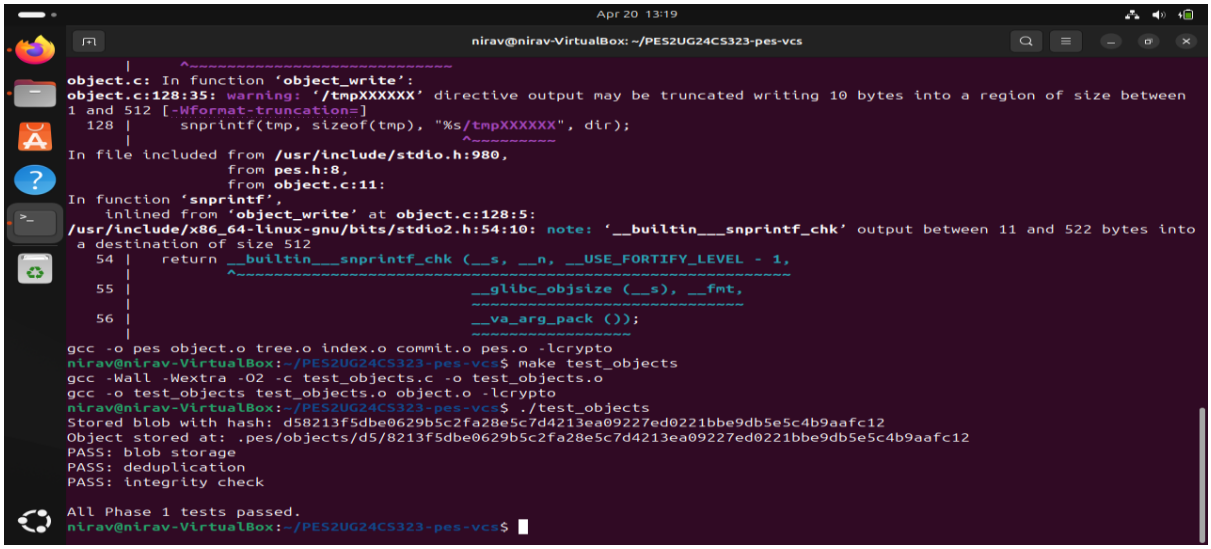
This project implements a simplified local version control system called PES-VCS. It supports object storage, staging files, creating commits, and viewing commit history similar to Git internals.

Implemented Source Files:

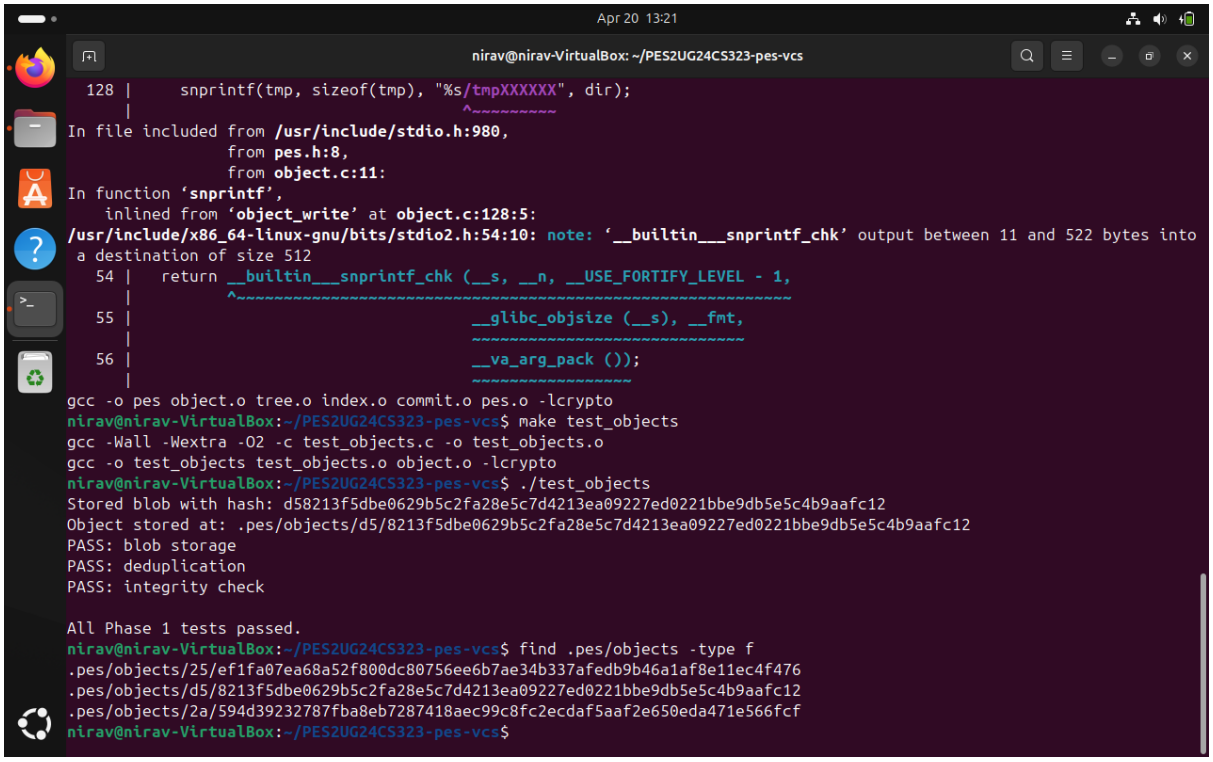
1. object.c – Content addressable object storage
2. tree.c – Tree object creation and serialization
3. index.c – Staging area operations
4. commit.c – Commit creation and history

SCREENSHOTS OF OUTPUTS

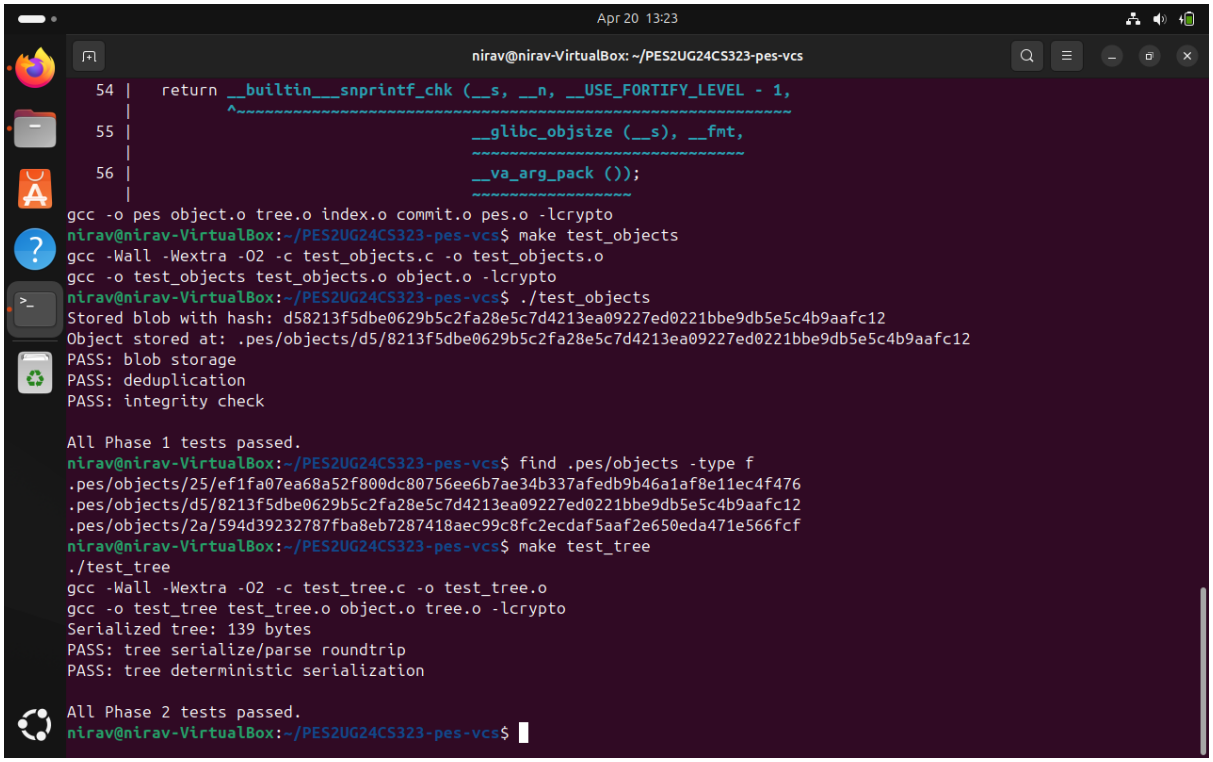
PHASE 1:



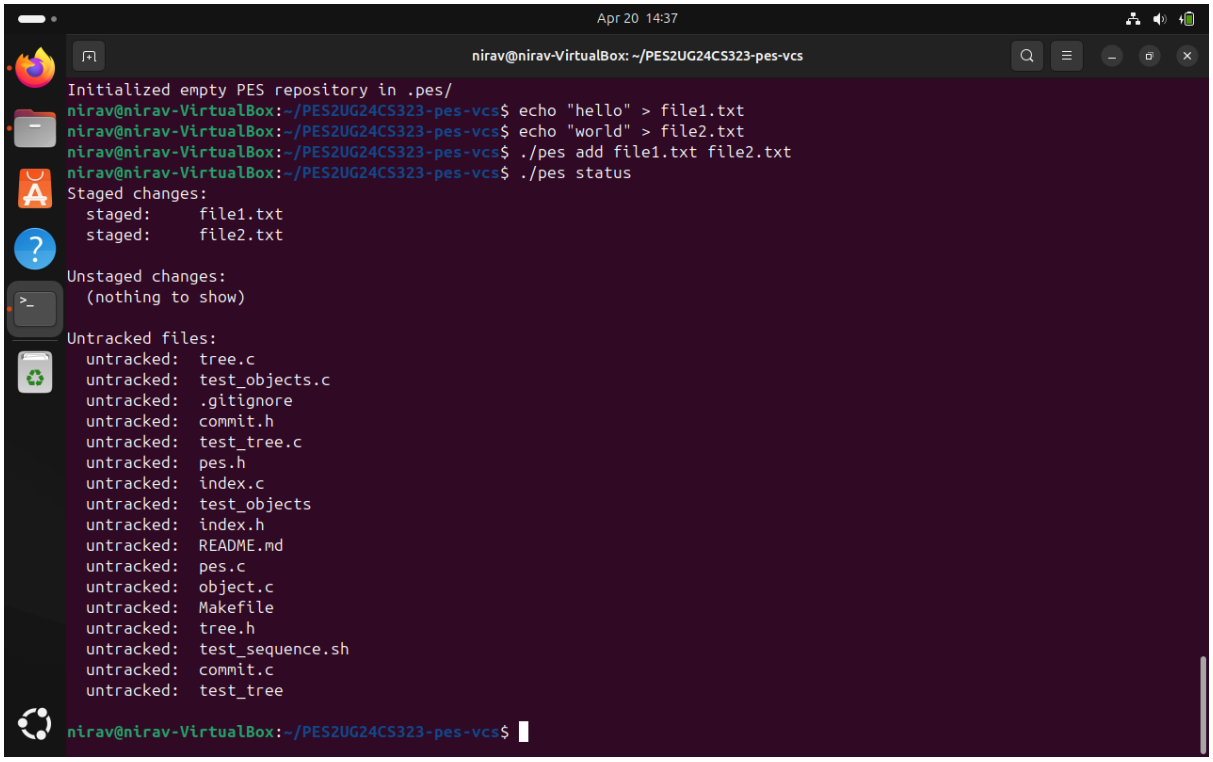
1A ./test_objects output showing all tests passing



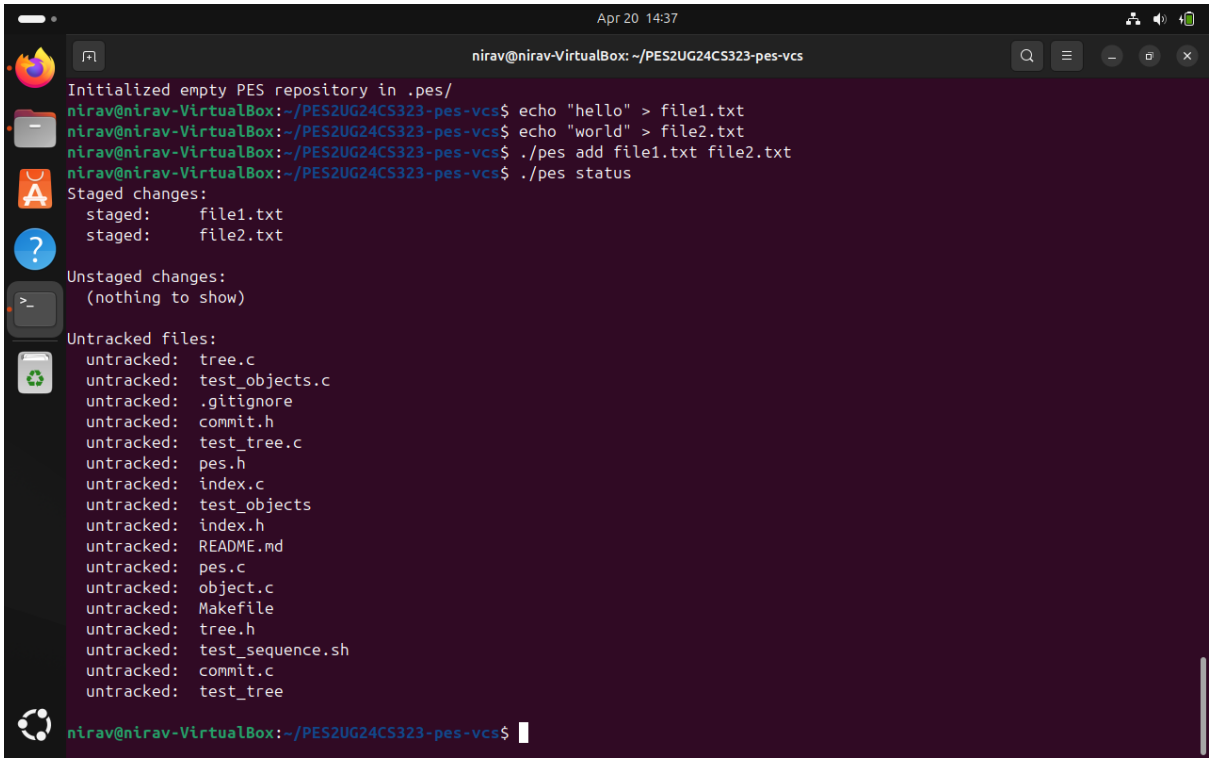
1B find .pes/objects -type f showing sharded directory structure



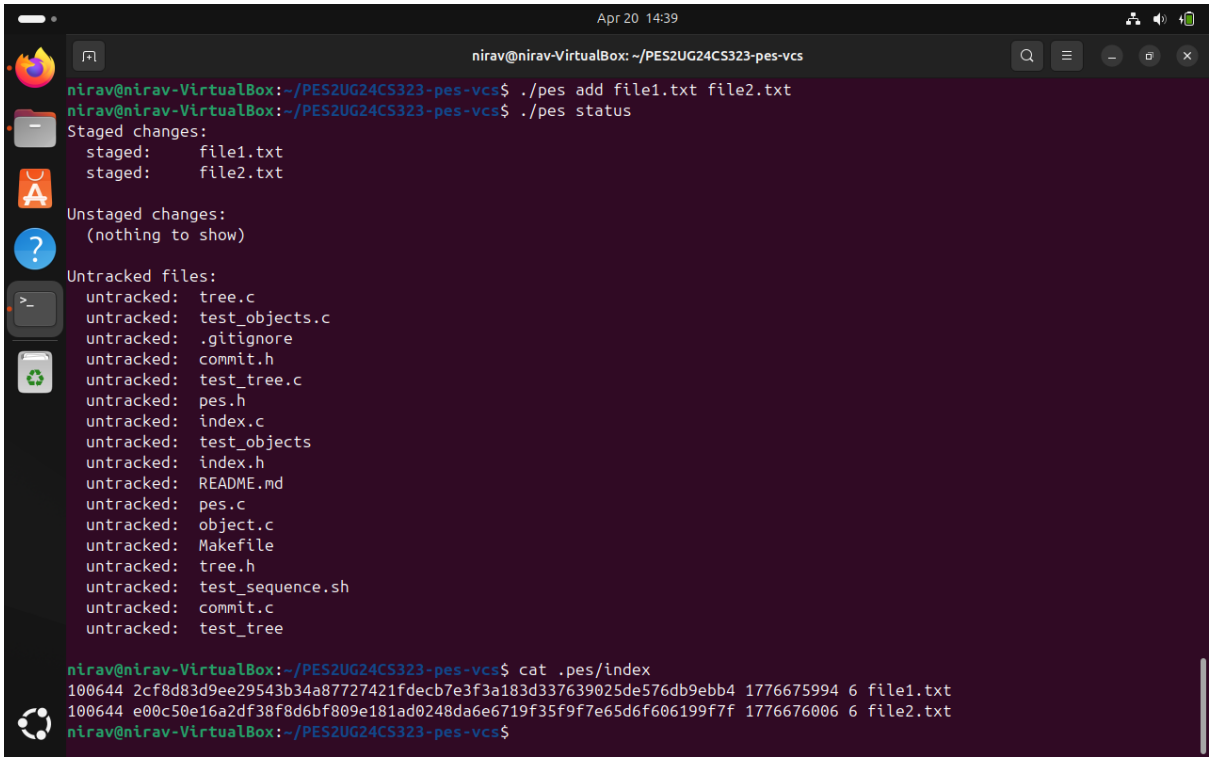
2A ./test_tree output showing all tests passing



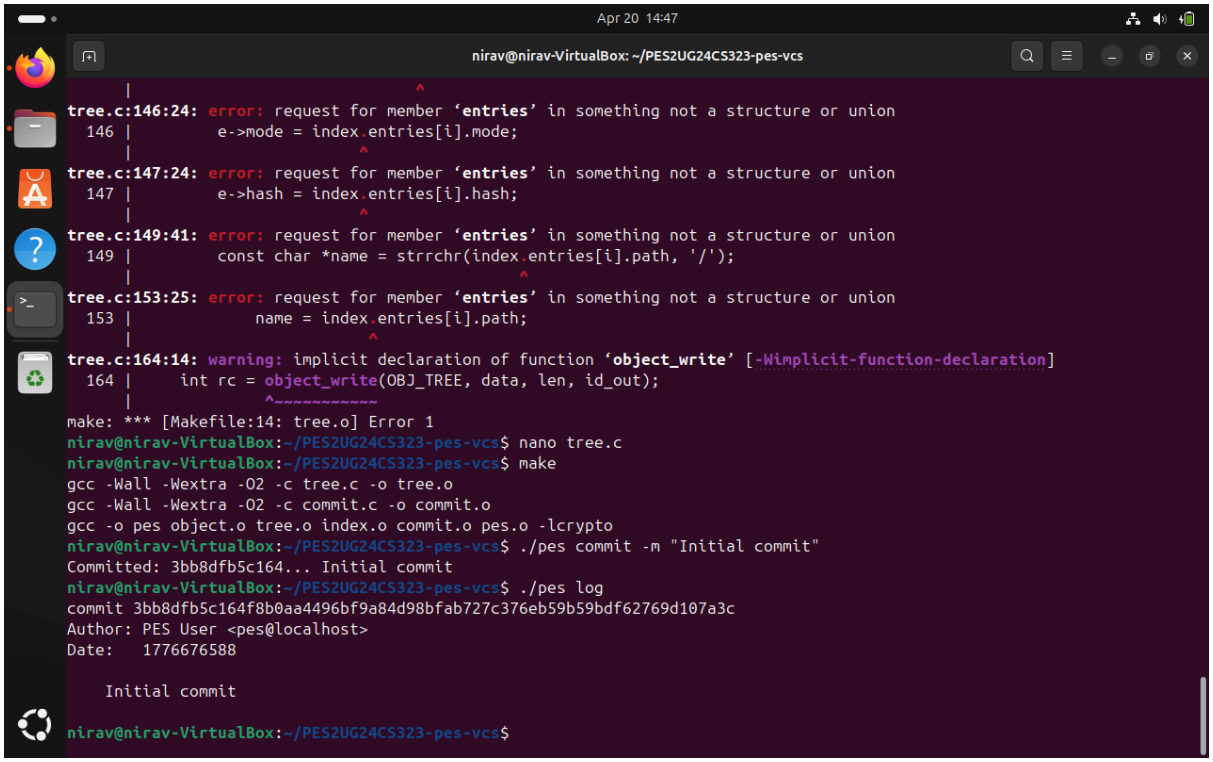
2B xxd of a raw tree object (first 20 lines)



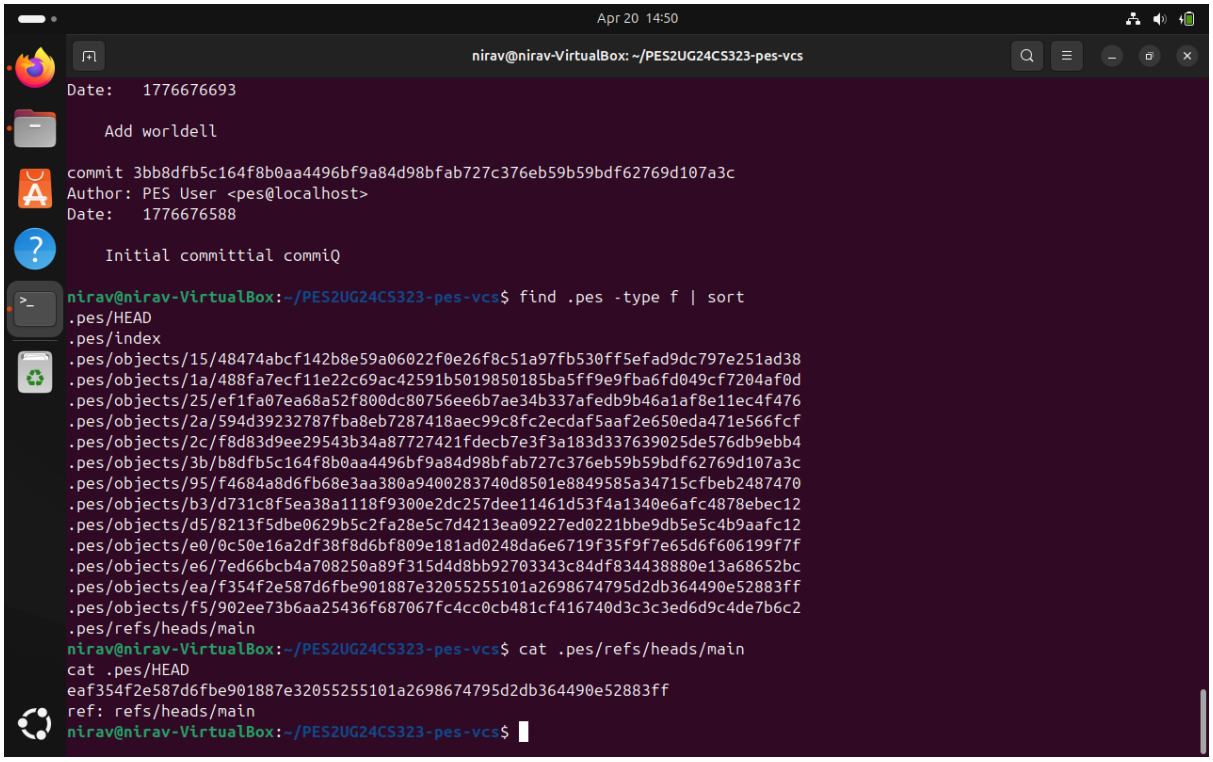
3 3A pes init → pes add → pes status sequence



3 3B cat .pes/index showing the text-format index



4A pes log output with three commits



find .pes -type f | sort showing object growth

cat .pes/refs/heads/main and cat .pes/HEAD

QUESTION AND ANSWERS

5.1) To implement pes checkout <branch>, first read the commit hash from .pes/refs/heads/<branch>. Then update .pes/HEAD so it points to that branch. After that, read the commit's tree object and update the working directory files to match that snapshot. Files not in the target branch should be removed, and changed files should be replaced. This operation is complex because local uncommitted changes may be overwritten and directories/files must be synchronized correctly.

5.2) To detect a dirty working directory, compare current files with entries stored in the index using file size, modified time, or hash. Then compare the target branch tree with the current branch tree. If a tracked file has local changes and that same file is different in the target branch, checkout should stop and refuse. This prevents loss of uncommitted work.

5.3) Detached HEAD means .pes/HEAD stores a commit hash directly instead of a branch name. If commits are made here, new commits are created but no branch moves to them, so they can become unreachable later. To recover them, create a new branch pointing to that commit hash before garbage collection removes them.

6.1) Start from all branch heads in .pes/refs/heads/ and mark every reachable commit, tree, and blob by traversing parent commits and tree references. Store visited hashes in a hash set for fast lookup. After marking, scan .pes/objects/ and delete any object not marked reachable. For 100,000 commits and 50 branches, around 100,000 commits plus related trees/blobs may need to be visited, depending on sharing.

6.2) Running garbage collection during a commit is dangerous because GC may delete a newly created tree or blob before the commit reference is fully updated. For example, commit writes objects first, but branch pointer is updated later. GC may think those objects are unreachable and delete them. Git avoids this using locks, temporary files, and by not deleting very recent objects during GC.

The project successfully demonstrated the internal working principles of a version control system including hashing, snapshots, indexing, commits, and history traversal.